Evaluación Técnica - Vacante Full Stack Web Engineer

Gracias por continuar en el proceso. Esta evaluación consta de **dos ejercicios prácticos** diseñados para evaluar calidad de código, criterio técnico y capacidad de diseño.

Tienes hasta el **1ero de marzo** para enviar las soluciones de ambos ejercicios.

Puedes usar JavaScript o TypeScript

Ejercicio 1 - Implementación de Rate Limiter en Memoria

Necesitamos implementar un mecanismo de rate limiting en memoria que limite el número de requests permitidos por usuario en una ventana de tiempo específica. Este ejercicio evalúa:

- Manejo de tiempo
- Elección de estructura de datos
- Limpieza y organización de código
- Pensamiento backend
- Consideración de edge cases

Implementa la siguiente función:

createRateLimiter(limit, windowMs)

Debe retornar una función que:

- Reciba un *userId*
- Permita máximo *limit* requests por *windowMs*
- Bloque (retorna *false*) si excede el límite
- Permita (retorna *true*) si está dentro del límite

Importante considerar:

- Debe funcionar completamente en memoria
- No usar Redis, base de datos ni librerías externas
- Debe soportar múltiples usuarios
- Debe considerar limpieza de registros viejos
- No es necesario manejar concurrencia real

Ejercicio 2 - Refactorización de componente React

Se te proporcionará un archivo funcional llamado ClientsTable.jsx que:

- Renderiza una tabla de datos
- Maneja filtros y ordenamiento
- Usea React Table
- Tiene lógica de expansión
- Implementa paginación
- Usa estado derivado y varios hooks

El componente funciona correctamente, pero presenta oportunidades de mejora en:

- Organización
- Separación de responsabilidades
- Performance
- Legibilidad
- Complejidad

Refactoriza el componente con los siguiente objetivos

1. Mejora legibilidad
2. Reducir complejidad cognitiva
3. Corregir posibles problemas en hooks
4. eliminar estado derivado innecesario

5. Separar responsabilidades cuando sea apropiado
6. mejorar nombre de variables y funcionales si aplica
7. Eliminar código muerto

Importante considerar:

- No cambiar la funcionalidad
- No cambiar comportamiento visible
- No eliminar features
- No agregar nuevas librerías
- Puedes:
    - Crear hooks personalizados
    - Extraer utilidades
    - Dividir en componentes
    - Usar TypeScript si deseas

Entregables:

Debes compartir tus soluciones subiendolas a un repositorio público en GitHub y enviarnos el enlace por correo. El README.md debe explicar decisiones técnicas tomadas, problemas encontrados y posibles mejoras futuras.

No buscamos perfección absoluta, sino claridad en tu criterio técnico y calidad en la implementación.

Si tienes cualquier duda o comentario durante la evaluación, puedes escribirnos por correo.